

1. What is Apache Kafka?

Apache Kafka is a **distributed event-streaming platform** designed to handle **huge volumes of data in real time**. It allows applications to **publish, store, process, and consume streams of records** efficiently.

Kafka is not just a messaging system. It acts like:

- A **message broker**
- A **distributed commit log**
- A **streaming backbone** for microservices

Kafka is widely used in systems where **data must never be lost**, must be **processed fast**, and should be **scalable horizontally**, such as order processing, payments, notifications, logs, and analytics.

2. Why do we use Kafka?

Kafka is used because traditional systems (DB polling, REST APIs, JMS queues) **fail at scale**.

Kafka solves problems like:

- High traffic spikes
- Real-time processing
- Loose coupling between services
- Reliable data delivery

Kafka can process **millions of messages per second**, store them safely, and allow multiple consumers to read the same data independently.

3. What are the main components of Kafka?

Kafka has several core components that work together:

1. **Producer** – Sends messages to Kafka
2. **Broker** – Kafka server that stores data
3. **Topic** – Logical category of messages
4. **Partition** – Physical split of a topic
5. **Consumer** – Reads messages
6. **Consumer Group** – Multiple consumers working together
7. **Offset** – Position of a message inside a partition

Each component plays a specific role in making Kafka scalable and fault-tolerant.

4. What is a Kafka Topic?

A topic is a **logical stream of messages** in Kafka.

Think of a topic as:

- A **table name** in a database
- Or a **folder** where related messages are stored

Example:

- order-topic
- payment-topic
- notification-topic

Producers write messages to topics, and consumers read messages from topics.

5. What is a Partition in Kafka?

A partition is a **physical unit of storage** inside a topic.

Each topic is split into one or more partitions to:

- Increase parallelism
- Improve performance
- Allow scalability

Each partition is an **ordered, immutable sequence of records**.

6. Why are partitions so important?

Partitions are the **heart of Kafka scalability**.

They allow:

- Multiple consumers to read data in parallel
- Data to be distributed across brokers
- High throughput

⚠ Important rule:

Message ordering is guaranteed only within a single partition, not across partitions.

7. What is a Kafka Broker?

A broker is a **Kafka server instance**.

Responsibilities of a broker:

- Store topic partitions on disk

- Serve producer write requests
- Serve consumer read requests
- Replicate data to other brokers

A Kafka cluster always contains **multiple brokers**.

8. What is a Producer?

A producer is a client application that **publishes messages** to Kafka topics.

Producer responsibilities:

- Choose the topic
- Decide partition (key-based or round-robin)
- Serialize data
- Handle acknowledgments and retries

Producers do **not talk to consumers directly**.

9. What is a Consumer?

A consumer is a client application that **reads messages** from Kafka topics.

Consumers:

- Pull data from Kafka brokers
- Track offsets
- Process messages independently

Kafka allows **multiple consumers to read the same data**, which is not possible in traditional queues.

10. What is a Consumer Group?

A consumer group is a **logical group of consumers** working together.

Rules:

- Each partition is consumed by **only one consumer within a group**
- Multiple groups can read the same topic independently

This enables:

- Load balancing
 - Fault tolerance
 - Scalability
-

11. What is an Offset?

An offset is a **unique sequence number** assigned to each message within a partition.

Offsets:

- Are strictly increasing
- Identify the position of a message
- Are managed by consumers, not brokers

Kafka stores offsets in an internal topic called `__consumer_offsets`.

12. Is Kafka push-based or pull-based?

Kafka is **pull-based**.

Consumers request data when they are ready.

This design:

- Prevents consumer overload
 - Handles back pressure naturally
 - Gives consumers full control
-

13. What does append-only log mean?

Kafka stores data in **append-only log files**.

This means:

- Messages are only added at the end
- No update or delete in between
- Sequential disk writes → very fast

This design is one of the main reasons Kafka is so fast.

14. Does Kafka delete messages after consumption?

No ❌

Kafka **does NOT delete messages after a consumer reads them**.

Messages are deleted based on:

- Time-based retention
- Size-based retention
- Log compaction

This allows replaying data anytime.

15. Kafka vs RabbitMQ (deep explanation)

Kafka and RabbitMQ solve **different problems**.

Kafka:

- Designed for streaming
- High throughput
- Pull model
- Data replay possible
- Distributed log

RabbitMQ:

- Message queue
- Low latency
- Push model
- Messages removed after consumption

Kafka is preferred for **event-driven architectures**.

● INTERMEDIATE LEVEL (16–35)

16. How does Kafka achieve high throughput?

Kafka achieves high throughput using:

- Sequential disk writes
- Zero-copy optimization
- Batch processing
- Partitioned parallelism
- Page cache usage

Kafka writes data to disk **faster than many systems write to memory**.

17. What is replication factor?

Replication factor defines **how many copies of a partition exist** across brokers.

Example:

- Replication factor = 3
- One leader + two followers

This ensures **fault tolerance and durability**.

18. Leader and Follower in Kafka

For each partition:

- One broker is the **leader**
- Others are **followers**

All reads and writes go to the leader.

Followers replicate data from the leader.

19. What happens if a broker goes down?

If a broker fails:

- Kafka elects a new leader from ISR
- Producers and consumers continue working
- No data loss (if replication exists)

Kafka is designed for **self-healing**.

20. What is ISR (In-Sync Replicas)?

ISR is the set of replicas that:

- Are fully synced with the leader
- Have latest data
- Are eligible for leader election

ISR ensures **data consistency and safety**.

21. How does Kafka guarantee message ordering?

Kafka guarantees ordering **within a partition only**.

If messages go to different partitions, ordering is not guaranteed.

Key-based partitioning is used when ordering matters.

22. What delivery semantics does Kafka support?

Kafka supports:

- At-most-once → no retries
- At-least-once → retries possible
- Exactly-once → no loss, no duplicates

Exactly-once requires idempotent producers and transactions.

23. How does a consumer know where to read from?

Consumers track offsets stored in Kafka's internal topic `__consumer_offsets`.

Offsets represent the **last committed position**.

24. What is consumer rebalance?

Consumer rebalance is the process where Kafka **redistributes partitions** among consumers in a group.

Occurs automatically.

25. When does rebalance occur?

Rebalance happens when:

- New consumer joins
- Consumer crashes
- Partition count changes

During rebalance, consumption pauses briefly.

26. What is auto offset commit?

Kafka automatically commits offsets at a configured interval.

Risk:

- Data loss if processing fails before commit
-

27. `enable.auto.commit` true vs false

- `true` → Kafka manages commits (simple but risky)
 - `false` → Application controls commits (safe, recommended)
-

28. What happens if a consumer crashes?

Kafka detects failure and:

- Triggers rebalance
 - Assigns partitions to another consumer
 - Continues processing from last committed offset
-

29. What is message retention policy?

Retention policy defines **how long messages are stored**, independent of consumption.

This enables reprocessing and debugging.

30. Retention time vs retention size

- Time-based → delete after X hours/days
 - Size-based → delete when log exceeds size
-

31. What is log compaction?

Log compaction keeps **only the latest value for each key**.

Useful for:

- User profile updates
 - Configuration data
 - State storage
-

32. What is idempotent producer?

Idempotent producer ensures **no duplicate messages** even if retries occur.

Enabled via:

`enable.idempotence=true`

33. What is acks=0,1,all

- 0 → fire and forget
 - 1 → leader ack
 - all → leader + replicas ack (safest)
-

34. Serializer and Deserializer

They convert:

- Object → byte[] (serializer)
- byte[] → object (deserializer)

Kafka stores everything as bytes.

35. What is a Dead Letter Topic?

A DLT stores messages that:

- Failed processing
 - Caused exceptions
 - Need manual investigation
-

● ADVANCED LEVEL (36–50)

36. What is Exactly-Once Semantics?

Exactly-once ensures:

- Message processed only once
- No duplicates
- No loss

Achieved using:

- Idempotent producers
 - Transactions
 - Exactly-once consumers
-

37. How does Kafka ensure fault tolerance?

Kafka uses:

- Replication
- ISR
- Leader election
- Persistent storage

Even multiple broker failures can be tolerated.

38. What is Zookeeper in Kafka?

Earlier Kafka used Zookeeper for:

- Metadata
 - Leader election
 - Cluster coordination
-

39. What is KRaft mode?

KRaft is Kafka's **new internal consensus system**, removing Zookeeper dependency.

Benefits:

- Simpler architecture
 - Faster startup
 - Better scalability
-

40. Kafka Streams vs Consumer API

Kafka Streams:

- High-level
- Stateful processing
- Exactly-once

Consumer API:

- Low-level
 - Manual control
-

41. What is Kafka Connect?

Kafka Connect is a framework to:

- Import data into Kafka
- Export data from Kafka

No coding required.

42. Source vs Sink Connector

- Source → External → Kafka
 - Sink → Kafka → External
-

43. What is Schema Registry?

Schema Registry manages schemas and ensures:

- Compatibility
 - Evolution
 - Data consistency
-

44. Avro vs JSON vs Protobuf

- Avro → Compact, schema-based
 - JSON → Readable, larger
 - Protobuf → Fast, binary
-

45. What is Zero-Copy in Kafka?

Zero-copy allows Kafka to send data **directly from disk to network**, avoiding memory copy and CPU overhead.

46. How does Kafka handle back pressure?

Kafka uses:

- Pull model
 - Consumer lag
 - Configurable fetch size
-

47. What is throttling in Kafka?

Throttling limits producer/consumer rate to:

- Protect brokers
 - Maintain cluster stability
-

48. What if producer is faster than broker?

Broker buffers data.

If buffer is full:

- Producer blocks
 - Retries
 - Or fails
-

49. How do you secure Kafka?

Kafka security includes:

- SSL encryption
 - SASL authentication
 - ACL authorization
-

50. Real-time Kafka use cases

- Order processing
- Payment systems
- Notifications
- Event-driven microservices
- Log analytics